上讲主要内容：

- AES结构：精确到不同参数
- AES扩散与混淆：不同操作与两者的关系
- AES效率与安全：有限域操作、两轮完成完整扩散
- AES与DES的对比：整体优于DES，补充情况类似，多处点有突破

《密码学基础》

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

第7讲：分组密码模式

(Operation Modes of Block Cipher)

授课：徐国爱

内 容 Contents

1 经典密码模式

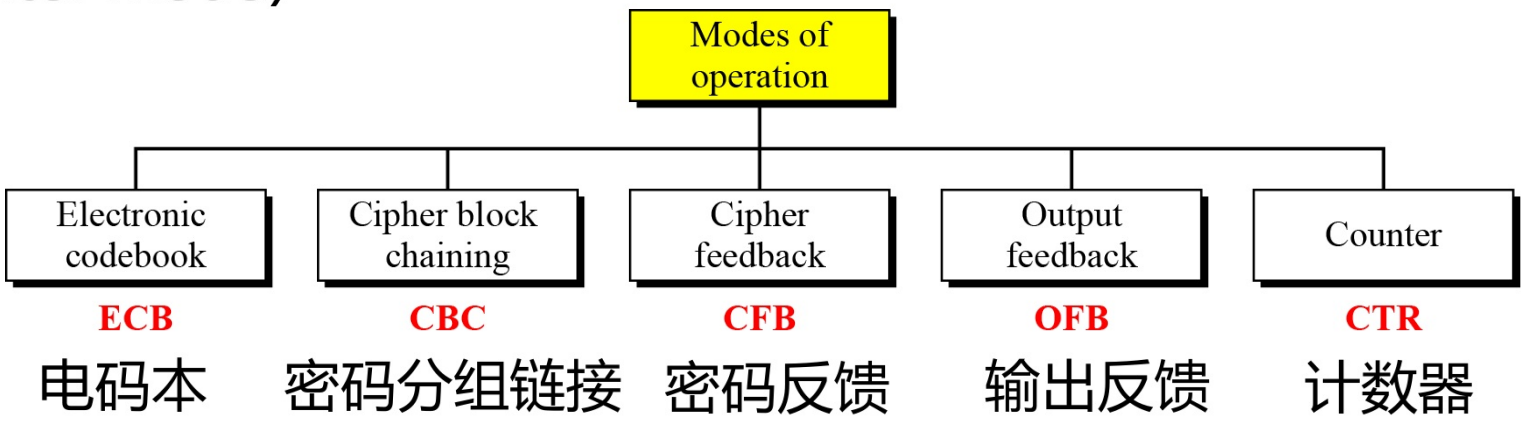
2 CBC模式安全

3 扩展密码模式

4 序列密码初步

1.1 分组密码工作模式分类

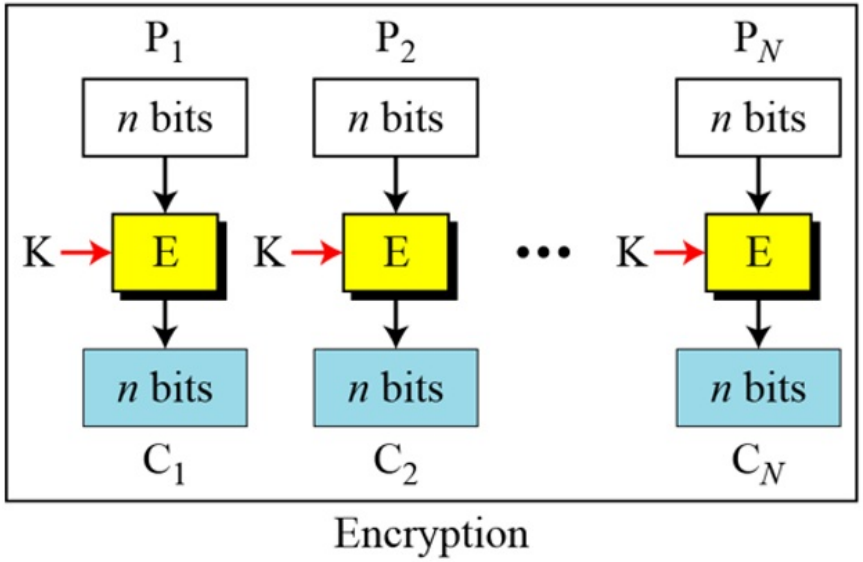
- 分组密码在加密时，明文分组长度固定，而实际应用中待加密消息的数据量不固定，数据格式可能多种多样
- 1980年NIST公布了DES的4种工作模式：电子密码本（Electronic Code Book，ECB）模式、密码分组链接（Cipher Block Chaining，CBC）模式、密码反馈（Cipher Feedback，CFB）模式和输出反馈（Output Feedback，OFB）模式。
- 2001年12月公布了AES的5种工作模式，即ECB、CBC、CFB、OFB和CTR（计数器模式，Counter Mode）



1.2电码本(ECB)模式

- 最简单的工作模式
- 加密：一次对一个64比特长的明文分组加密，每次加密密钥相同
- 适用于短消息加密传输

$$\boldsymbol{Encryption}: \boldsymbol{C_i = E_k(P_i)}$$



$$\boldsymbol{Decryption}: \boldsymbol{P_i = D_k(C_i)}$$

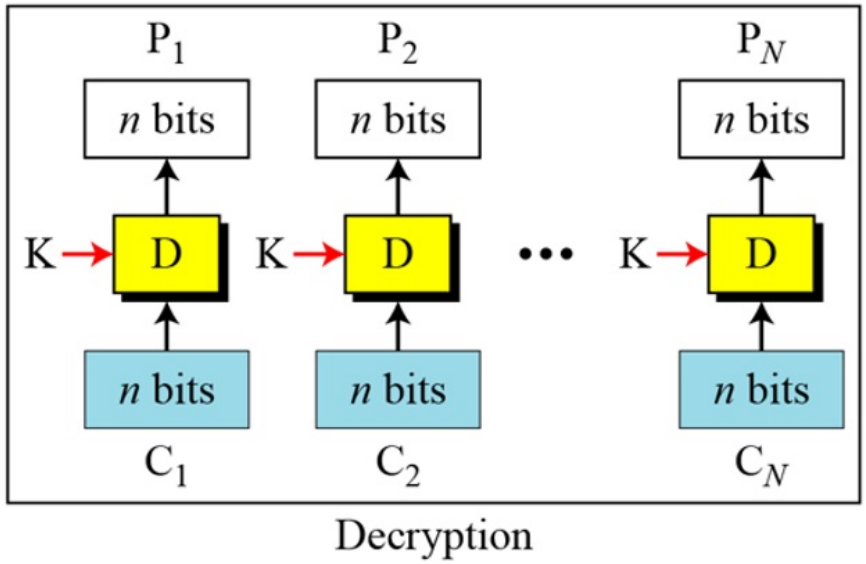


Figure 2 *Electronic codebook (ECB) mode*

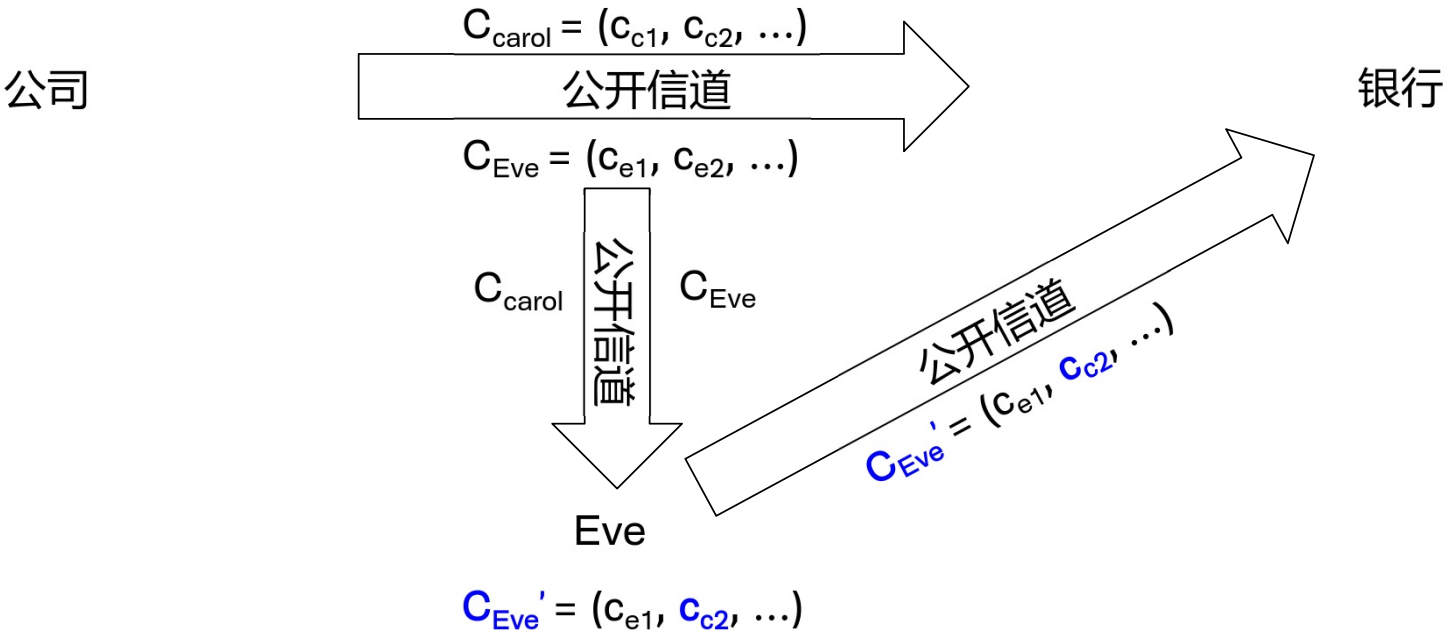
$E: Encryption \quad D: Decryption \quad K: Secret\ key$

$P_i: Plaintext\ block\ i \quad C_i: Ciphertext\ block\ i$

1.2 电码本(ECB)模式特点

- 在给定的密钥下，同一明文组总产生同样的密文组。不能抵抗重传攻击！

  - ✓ Eve知道工资密文中第2个分组是工资数额。
  - ✓ Eve截获高工资员工Carol工资密文，将其第2个分组替换自己的第2个分组



1.2 电码本(ECB)模式特点

- 在给定的密钥下，同一明文组总产生同样的密文组。**一定程度暴露明文数据的格式和统计特征**
  - ✓ The Penguin Principle: If you can still see the penguin after "encrypting" the image something is very very wrong with the encryption scheme.

Original image

1.2 ECB小结

- 优点：
  - ✓ 实现简单
  - ✓ 不同明文分组的加密可并行实施，尤其是硬件实现时速度很快
- 缺点：
  - ✓ 相同明文分组对应相同密文分组，不能隐蔽明文分组的统计规律和结构规律
  - ✓ 不能抵抗替换攻击

1.3 密码分组链接(CBC)模式

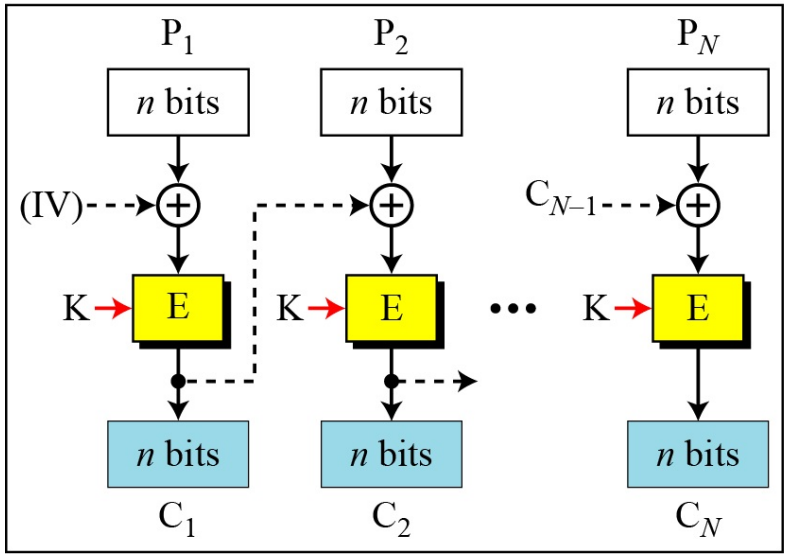
- 加密：一次对一个明文分组加密，每次加密使用同一密钥，加密算法的输入是当前明文分组和前一次密文分组的异或

- 密文与之前所有明文分组关联；明文仅与当前及前一个密文分组相关，可实现密文随机解密读取。

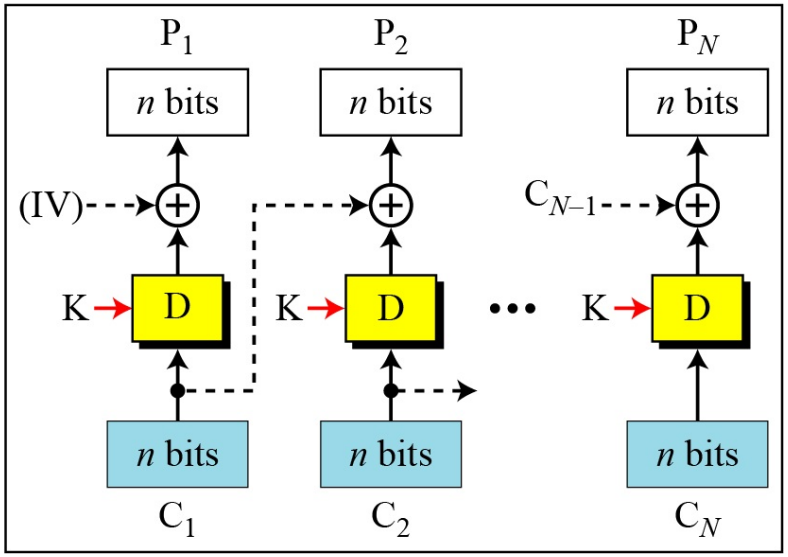
Encryption:	**Decryption**:
$C_0 = IV$	$C_0 = IV$
$C_i = E_k(P_i \oplus C_{i-1})$	$P_i = D_k(C_i) \oplus C_{i-1}$

Figure 3 *Cipher block chaining (CBC) mode*

E: Encryption　　D : Decryption
P_i: Plaintext block *i*　　C_i : Ciphertext block *i*
K: Secret key　　IV: Initial vector (C_0)



Encryption



Decryption

1.3 CBC初始矢量IV(Initialization Vector)

- 第一组明文$\boldsymbol{P_1}$加密时尚无反馈密文，为此需要在寄存器中预先置入一个IV（由随机比特组成的分组）
- 收发双方必须选用同一IV
- IV对于收发双方都应是已知的。不需要保密，但要保障完整性

1.3 CBC要点-填充

	BLOCK #1								BLOCK #2							
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8
Ex 1	F	I	G													
Ex 1 (Padded)	F	I	G	0x05	0x05	0x05	0x05	0x05								
Ex 2	B	A	N	A	N	A										
Ex 2 (Padded)	B	A	N	A	N	A	0x02	0x02								
Ex 3	A	V	O	C	A	D	O									
Ex 3 (Padded)	A	V	O	C	A	D	O	0x01								
Ex 4	P	L	A	N	T	A	I	N								
Ex 4 (Padded)	P	L	A	N	T	A	I	N	0x08	0x08	0x08	0x08	0x08	0x08	0x08	0x08
Ex 5	P	A	S	S	I	O	N	F	R	U	I	T				
Ex 5 (Padded)	P	A	S	S	I	O	N	F	R	U	I	T	0x04	0x04	0x04	0x04

At least one padding byte is ALWAYS appended

1.3 CBC要点-优缺点

- 优点：
  - ✓ 能隐蔽明文的数据模式，链接的方式在某种程度上能防止数据篡改，诸如重放、嵌入和删除等。
  - ✓ 可以用于构造消息认证码进行数据完整性认证
- 缺点：CBC的错误传播
  - ✓ 明文有一组中有错，会使以后的密文组都受影响，但经解密后的恢复结果，除原有误的一组外，其后各组明文都正确地恢复。
  - ✓ 若传送过程中，某组密文组C_i出错时，则该组恢复的明文P'_i和下一组恢复数据P'_{i+1}出错。往后的组将不会受C_i中错误比特的影响

Contents 内 容

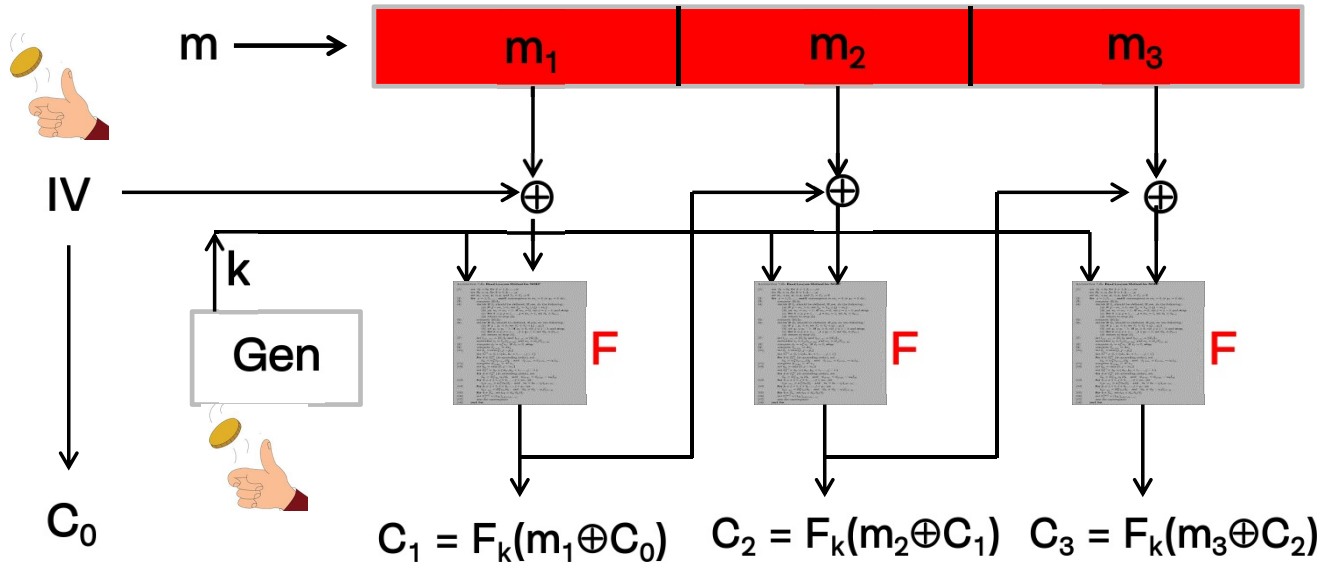
1 经典密码模式

2 CBC模式安全

3 扩展密码模式

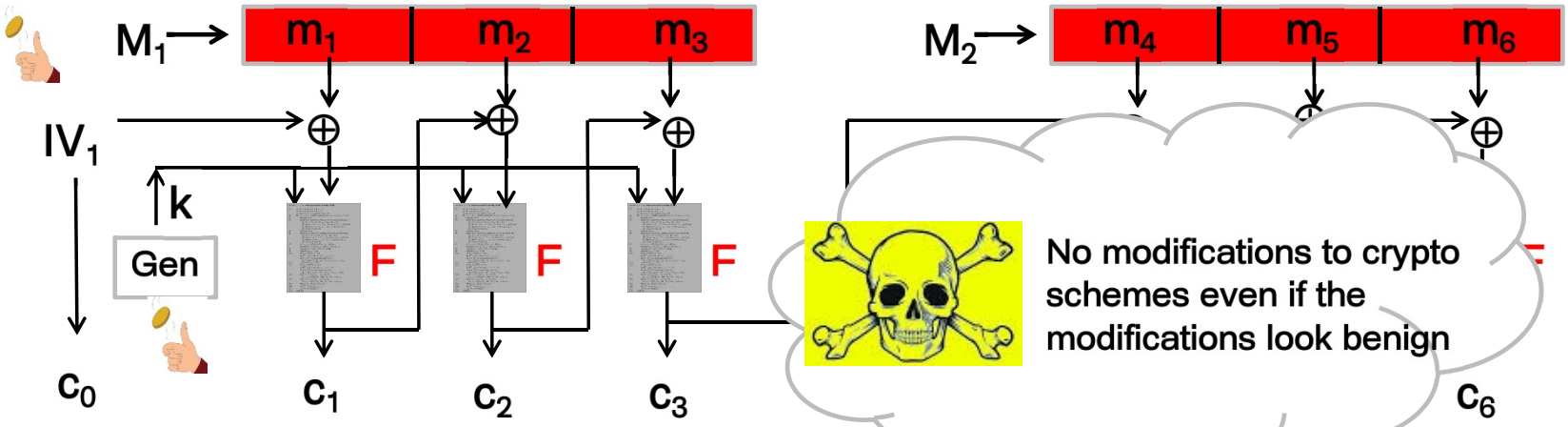
4 序列密码初步

2.1 错误使用IV-例1-猜测初始化向量



- Choosing distinct IV enough ? Can save randomness
- Unfortunately this version of CBC mode is not cpa-secure.
- Attack? IV = 1, then increment IV by one

2.1 错误使用IV-例2-猜测cookie



- Chained CBC mode --- used in SSL 3.0 and TLS 1.0

BEAST attack on SSL/TSL

- C

>> It is "equivalent" to encrypting a single large message $M = M_1 \| M_2$ via CBC mode

>> Yet NOT CPA-secure

>> Send m_0 + IV + c in the post-challenge training phase

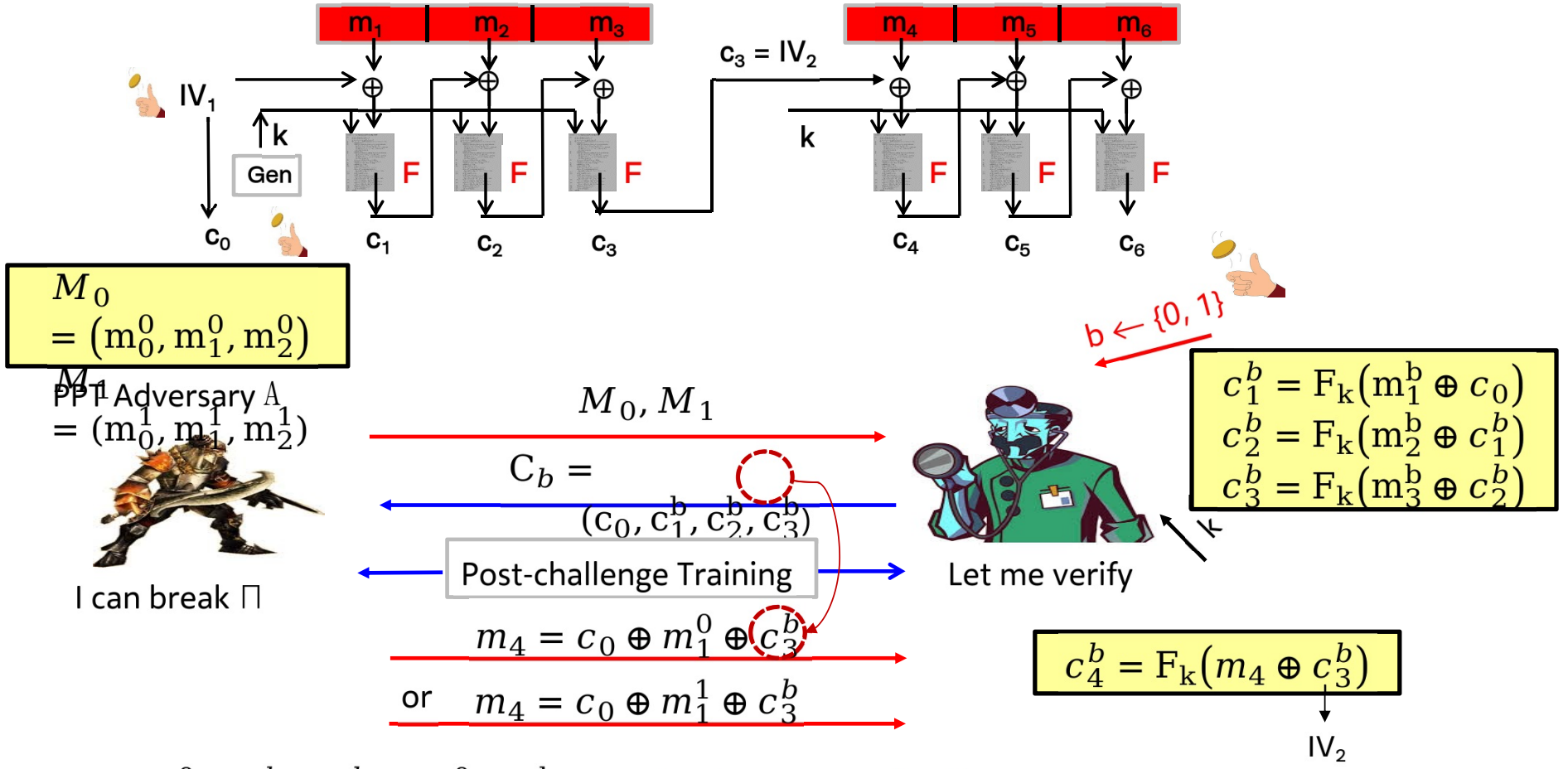
二、核心数学推导（攻击核心）

攻击者已知下一轮加密的IV（记为 IV_{known}）。
攻击者**可控构造明文前缀** P_{atk}，让浏览器加密：
$C_{atk} = E_K(P_{atk} \oplus IV_{known})$

目标：把**待猜测的秘密字节（Cookie）**放到明文块末尾。
攻击者不断猜测秘密字节 b，调整构造的 P_{atk}，使得：
$P_{atk} \oplus IV_{known} = (\text{固定前缀})||b$
浏览器加密后生成密文 C_{atk}。
攻击者对比密文，如果匹配，就代表猜测的字节 b 正确。

> 不需要破解密钥 K；只需要不断试字节，**一次恢复1字节Cookie。**

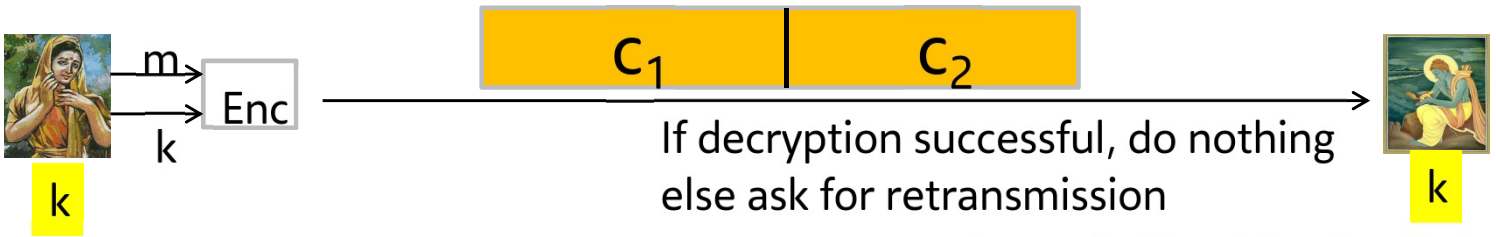
2.1 错误使用IV-例2-1比特挑战



If $c_4^b = F_k(c_0 \oplus m_1^0 \oplus c_3^b \oplus c_3^b) = c_1^0 = c_1^b$, then b = 0 ,otherwise b = 1

or If $c_4^b = F_k(c_0 \oplus m_1^1 \oplus c_3^b \oplus c_3^b) = c_1^1 = c_1^b$, then b = 1 ,otherwise b = 0

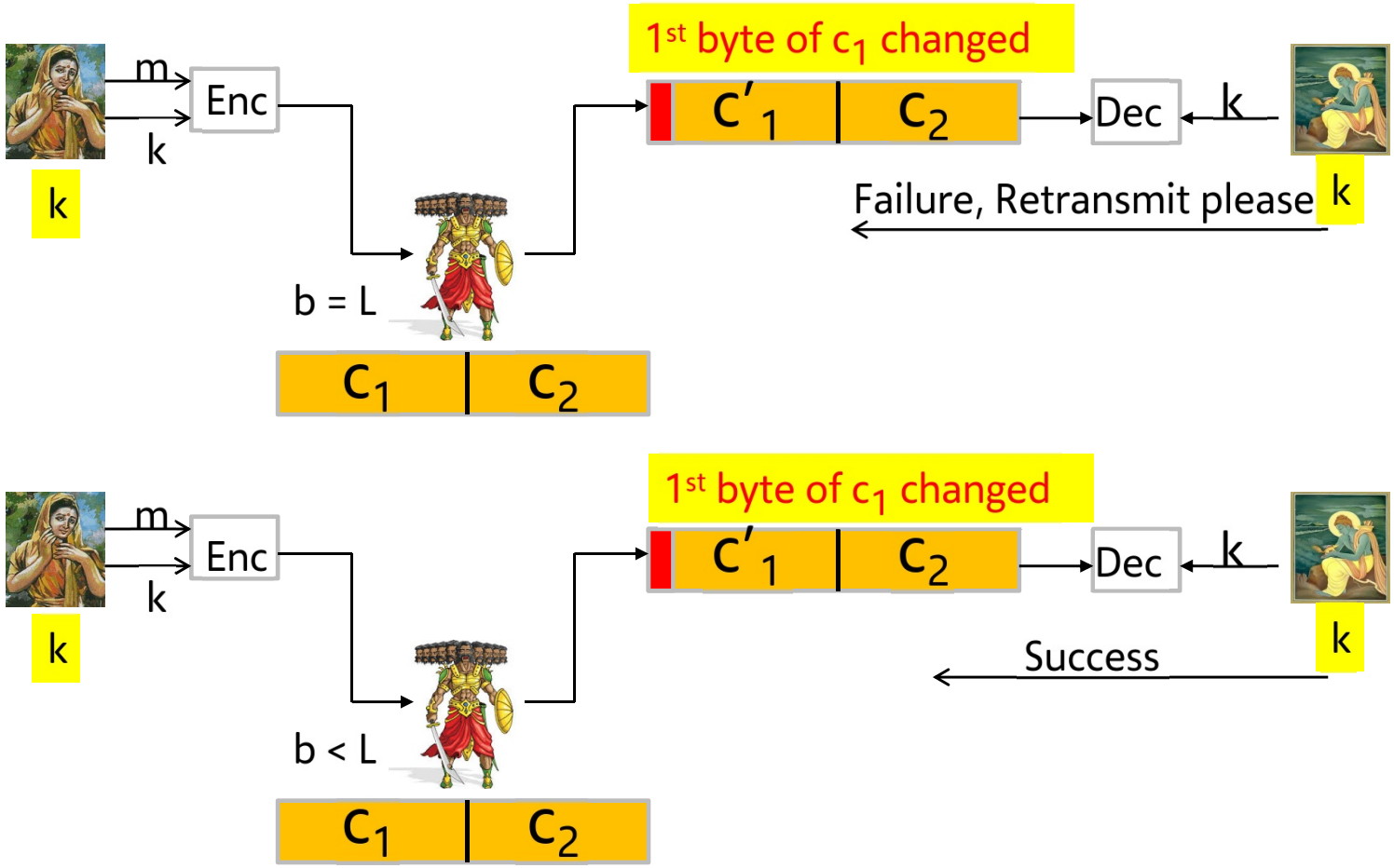
2.2 CBC填充攻击-填充长度识别



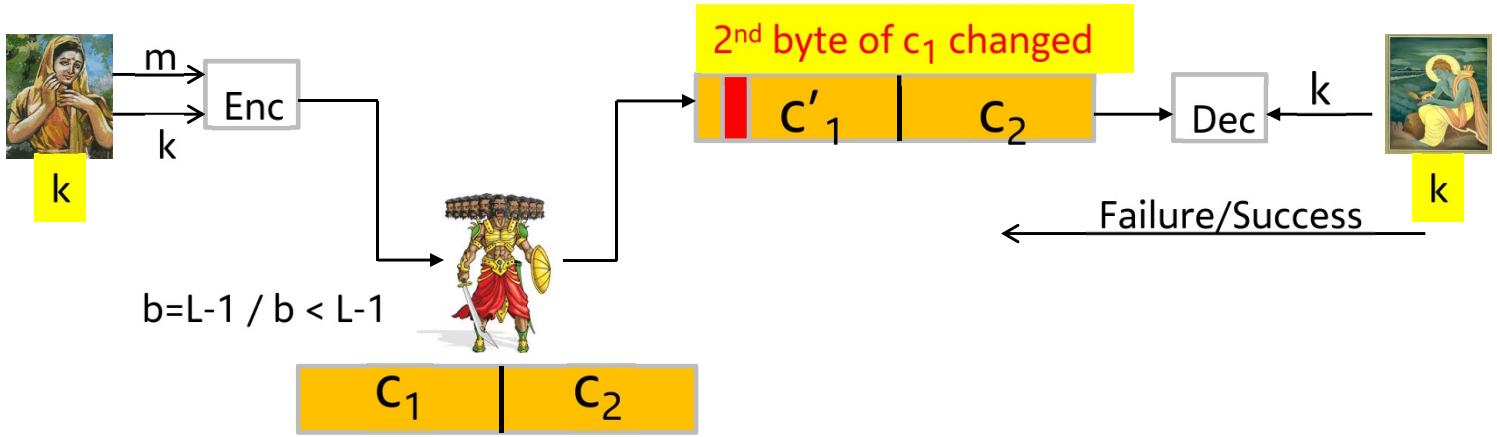
- An attacker can modify the ciphertexts and learn b (|m| leaked) and m.
- Hint: What will happen to the decryption of m_2 if the i^{th} byte of c_1 is modified by Δ ?
  - m_2' on decryption will be modified by Δ at i^{th} byte !!

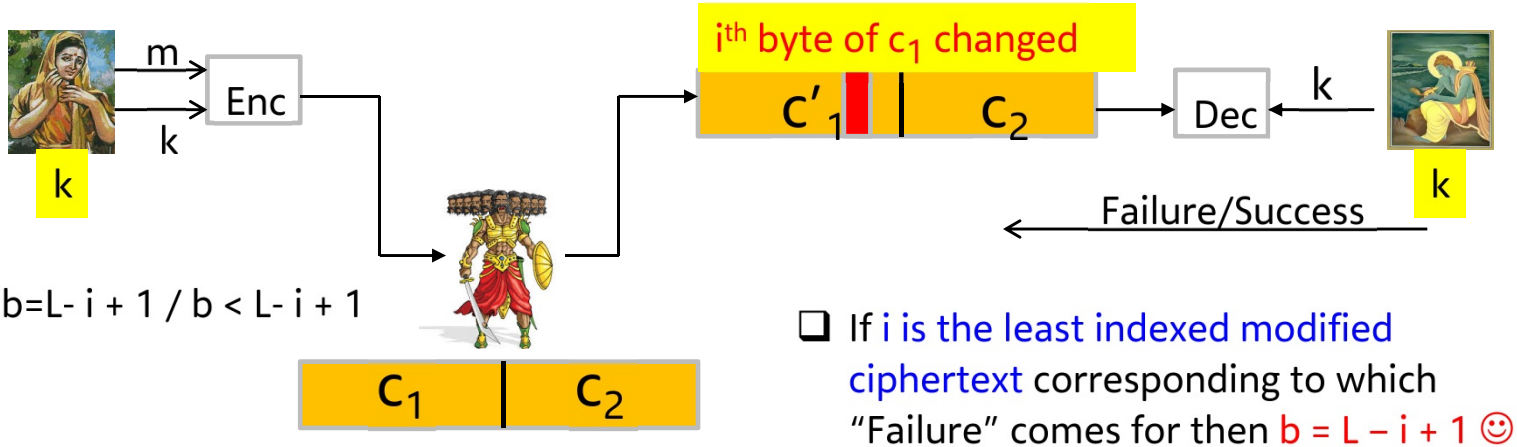
$$m_2 = F_k^{-1}(c_2) \oplus c_1 \qquad c_1' = c_1 \oplus \Delta$$
$$m_2' = F_k^{-1}(c_2) \oplus c_1' \qquad m_2' = m_2 \oplus \Delta$$

2.2 CBC填充攻击-填充长度识别



2.2 CBC填充攻击-填充长度识别

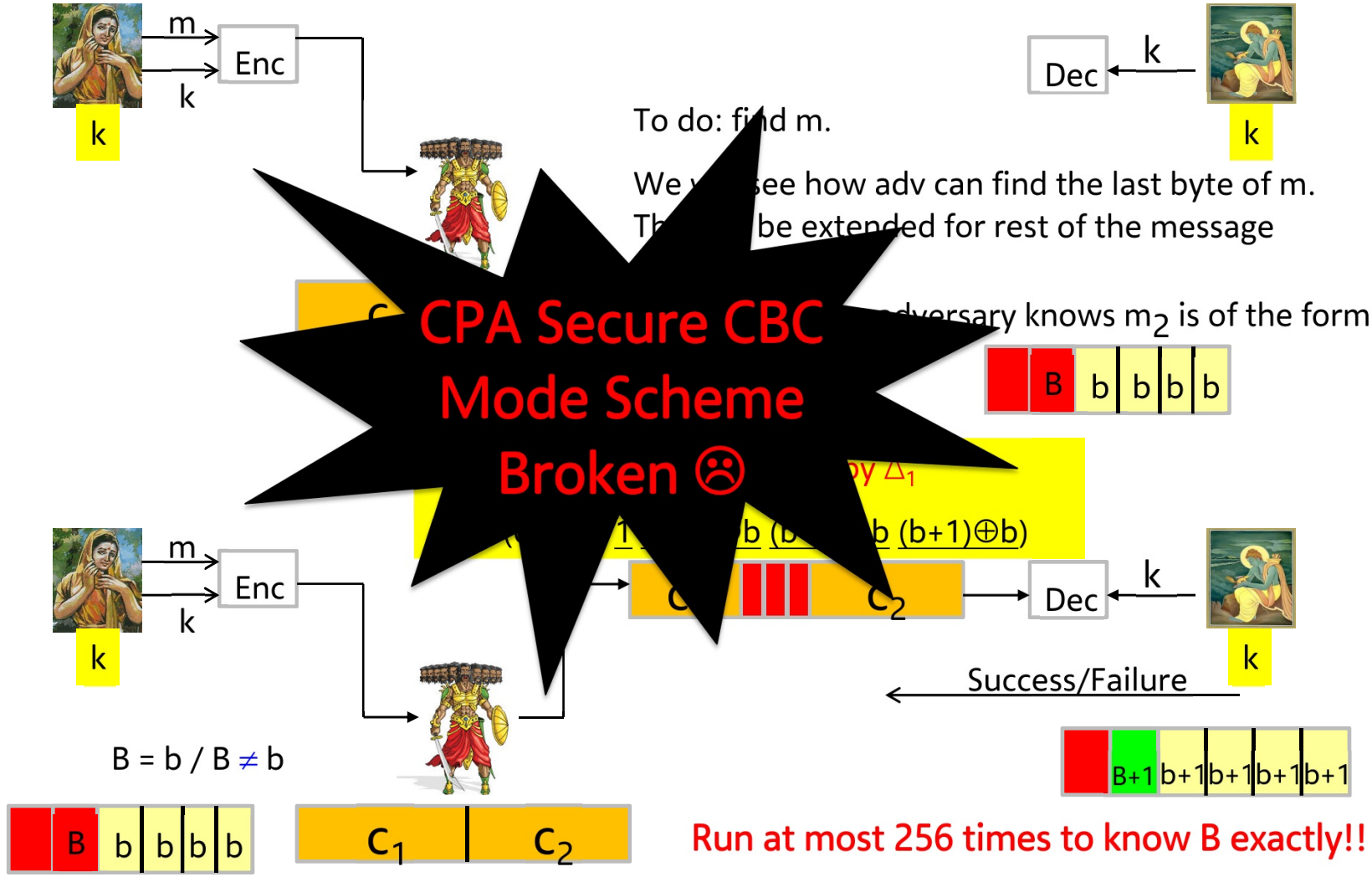




- If i is the least indexed modified ciphertext corresponding to which "Failure" comes for then b = L – i + 1 ☺

b is leaked. |m| is leaked!!

2.3 CBC填充攻击-逐比特还原明文!



目录

CONTENTS

01 **CBC 模式与填充基础**

CBC 加密原理 · PKCS#7 填充规则

02 **填充 Oracle 的侧信道本质**

错误响应差异 · 信息泄露通道

03 **攻击数学原理**

C_prev 控制明文 · 异或运算推导

04 **逐字节解密过程**

从最后一个字节开始 · 迭代还原

05 **攻击前提与经典案例**

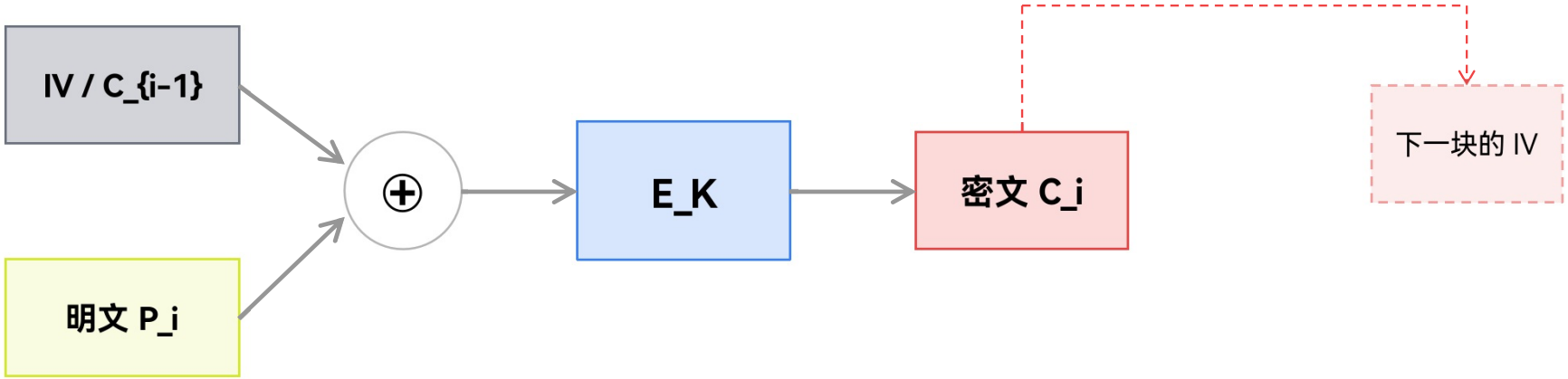
POODLE · Lucky13 · 真实影响

06 **防御措施与总结**

AEAD 加密 · 统一错误响应 · 最佳实践

CBC 模式：明文块与前一密文块异或后加密

$$C_i = E_K(P_i \oplus C_{i-1}), \quad C_0 = IV$$



关键洞察： C_{i-1} 是公开的密文，攻击者可以直接读取并任意修改。

修改 C_{i-1} 就能间接控制 P_i 解密后的结果——这是填充攻击的基石。

PKCS#7 填充：缺 n 字节就填 n 个值为 n 的字节

填充规则

块密码要求明文长度是块大小的整数倍(AES 为 16 字节）。
若最后一块缺 **n** 个字节，则填充 **n** 个值均为 **n** 的字节。
若明文恰好是块大小整数倍，则额外填充一整块(16 个 0x10）。

示例：明文 "HELLO"(5 字节），块大小 16 字节

解密后的填充验证

Step 1：读取最后一个字节的值 n

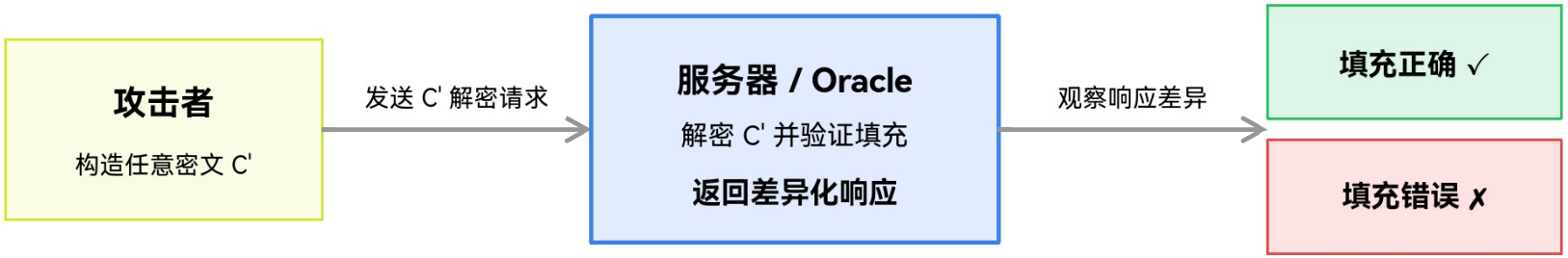
n 必须在 1 到块大小(16）之间

Step 2：检查最后 n 个字节是否都等于 n

全部等于 n → 填充有效；否则 → 填充错误

漏洞根源：服务器对"填充有效"和"填充错误"返回不同响应(错误码 / 耗时差异），攻击者可据此判断填充是否正确。

Padding Oracle：服务器充当"填充是否正确"的回答器



侧信道的三种常见形式

1. 错误码差异

填充错误返回 500 / 特定错误页；填充正确返回 200 / 正常响应。最直接、最易利用。

2. 响应时间差异

填充错误时提前终止、响应更快；填充正确时继续处理、响应更慢。Lucky13 攻击即利用此。

3. 连接行为差异

填充错误时服务器主动断开 TLS 连接；填充正确时保持连接。POODLE 攻击利用 SSLv3 的此特性。

修改前一密文块 C_{i-1}，即可控制解密后明文 P_i 的每一字节

$$P_i = D_K(C_i) \oplus C_{i-1}$$

1 **攻击者将 C_{i-1} 篡改为 C'_{i-1}，与原始 C_i 一起发给服务器解密**

服务器返回：$P'_i = D_K(C_i) \oplus C'_{i-1}$，然后验证 P'_i 的填充是否合法

2 **暴力枚举 C'_{i-1} 的最后一个字节（0~255），直到服务器返回"填充正确"**

填充正确意味着 P'_i 的最后一个字节 = 0x01（最简单的合法填充：1 个值为 1 的字节）

3 **由已知的 C'_{i-1} 推算出原始明文 P_i 的最后一个字节**

$$P_i[\text{last}] = D_K(C_i)[\text{last}] \oplus C_{i-1}[\text{last}] = (0x01 \oplus C'_{i-1}[\text{last}]) \oplus C_{i-1}[\text{last}]$$

全程不需要知道密钥 K，仅需 256 次尝试即可恢复 1 个明文字节

从最后一个字节开始，利用"递增填充"逐字节还原整个明文块

定义中间值 $I = D_K(C_i)$，则 $P_i = I \oplus C_{i-1}$。一旦通过攻击得到 I 的某字节，就能任意控制该位置的解密输出。

1 **恢复字节 16(最后一个)**

枚举 C'_{15}[16]，目标填充 = [0x01]。得到 I[16] = 0x01 ⊕ C'_{15}[16]

2 **恢复字节 15(倒数第二个)**

设 C'_{15}[16] = I[16] ⊕ 0x02 使字节16输出0x02；枚举字节15，目标填充 = [0x02, 0x02]

3 **恢复字节 14 ~ 2**

每次将已恢复字节设为目标填充值 n，枚举下一个字节，目标填充为 n 个 0x0n

4 **恢复字节 1(第一个)**

全部 16 字节恢复后，P_i = I ⊕ C_{i-1}，逐块解密完整密文

解密进度示意(16 字节块)

恢复方向：从右向左(字节 16 → 字节 1)

攻击复杂度

每字节：平均 128 次请求，最多 256 次

每 16 字节块：约 2048 次请求

1KB 密文：约 13 万次请求，数分钟内完成

攻击成立的四个必要前提，缺一不可

1. 攻击者处于中间人位置

能够拦截受害者与服务器之间的网络流量，读取密文并篡改后转发。公共 WiFi、ARP 欺骗、恶意代理等场景均可实现。

2. 加密使用 CBC 模式

前一密文块参与当前块解密运算，使攻击者可通过修改 C_{i-1} 控制 P_i。GCM、CCM 等 AEAD 模式不受此攻击影响。

3. 填充 Oracle 存在（核心）

服务器对"填充正确"和"填充错误"返回可区分的响应：不同错误码、不同响应时间、不同连接行为。这是攻击成立的最关键条件。

4. 密文可被重复发送

攻击者能向服务器反复发送构造的密文（每字节最多 256 次）。若服务器有严格的重放保护或请求频率限制，攻击难度会显著增加。

历史上三次影响深远的填充 Oracle 攻击

攻击名称	年份 / CVE	攻击对象	原理与影响
POODLE	2014 CVE-2014-3566	SSLv3 + CBC	通过协议降级迫使客户端使用 SSLv3；SSLv3 不验证填充字节内容，仅检查填充长度，攻击者逐字节窃取 HTTPS Cookie。导致主流浏览器全面禁用 SSLv3。
Lucky13	2013 CVE-2013-0169	TLS 1.0/1.1 + CBC	时序侧信道攻击：填充错误时 MAC 验证被跳过，响应更快；填充正确时执行 MAC 计算，响应更慢。通过精确测量响应时间区分填充是否正确，影响所有当时的 TLS CBC 实现。
ASP.NET Padding Oracle	2010 CVE-2010-3332	ASP.NET ViewState	ASP.NET 对加密 ViewState 的填充错误返回不同 HTTP 错误码，构成填充 Oracle。攻击者可解密任意 ViewState 并伪造合法 ViewState，绕过身份验证和 CSRF 防护。影响全球数百万 ASP.NET 网站。

三次攻击的共同根源：CBC 模式 + 填充验证的差异化响应

四层防御：从协议根因到应用实践，彻底消除填充 Oracle

1. 使用 AEAD 加密模式（根本解决）

AES-GCM、ChaCha20-Poly1305 等 AEAD 模式将加密与认证合为一体，不存在独立的填充验证步骤，从根本上消除填充 Oracle。TLS 1.3 已强制使用 AEAD。

2. Encrypt-then-MAC（先认证后解密）

先验证 MAC，MAC 不通过直接拒绝，不进入解密和填充验证流程。TLS 1.2 可选扩展支持，TLS 1.3 默认采用。即使仍用 CBC，也能阻断填充 Oracle。

3. 统一错误响应（消除侧信道）

填充错误与其他解密错误返回完全相同的响应：相同错误码、相同响应时间（填充错误时也执行 dummy MAC 计算）、相同连接行为。让攻击者无法区分填充是否正确。

4. 禁用旧协议 + 使用成熟加密库

服务器禁用 SSLv3、TLS 1.0，优先 TLS 1.2/1.3 与 AEAD 套件。应用层使用 libsodium、OpenSSL 等经过审计的加密库，切勿自行实现加密或填充逻辑。

BEAST 与 Padding Oracle：两种 CBC 攻击的核心区别

对比维度	BEAST 攻击	Padding Oracle 攻击
攻击类型	选择明文攻击 (CPA)	选择密文攻击 (CCA)
利用漏洞	TLS 1.0 CBC 中 IV 可预测（前一记录尾块）	填充验证的差异化响应构成侧信道
攻击者能力	注入可控明文 + 监听密文（只读）	篡改密文 + 观察服务器响应（读写）
攻击目标	恢复会话 Cookie 等特定明文	解密任意密文，甚至伪造合法密文
修复方式	TLS 1.1+ 每条记录使用独立随机 IV；1/n-1 记录拆分（临时缓解）	AEAD 加密模式；Encrypt-then-MAC；统一错误响应消除侧信道

共同点：均针对 CBC 模式；最终解法都是迁移到 AEAD（TLS 1.3）

2.3 针对填充攻击的改进建议

- 有鉴于此，要预防下一个密文填充攻击安全漏洞，密码学家和网络信息安全专家们建议从以下几个方面改进和更新：
- 采用认证加密方法，如具有最高安全性的Encrypt-then-MAC（EtM）：首先对明文进行加密，然后根据得到的密文生成MAC，密文和它的MAC一起发送；对密文的随意改动将造成消息认证错误，会被马上抛弃。比如SSHv2支持的hmac-sha1-etm@openssh.com算法。或者直接应用带有关联数据的认证加密（Authenticated Encryption with Associated Data，AEAD），既能保护数据私密性，又可以同时认证密文和明文。
- 弃用TLS中的所有CBC密码套件（Cipher Suite），阻断密文填充攻击的可能性。如内容分发网络（Content Delivery Network，CDN）服务提供商Cloudflare从2016年开始全面转向支持分组加密AES-GCM和流加密ChaCha20-Poly1305，两者都是基于AEAD的密码套件。
- 升级到TLS 1.2或TLS 1.3，禁用之前版本的SSH/TLS协议，同时禁止降级协商，强制安全性高于互通性。TLS 1.3不允许任何非AEAD加密算法。2021年3月，IETF发布当前最佳实践RFC 8996，正式宣布弃用 TLS 1.0 和 TLS 1.1。根据SSL Labs2021年9月的扫描统计，在全球最流行150,000个启用SSL/TLS的网站中99.6%支持TLS 1.2，48.9%的站点支持TLS 1.3。

2.4 填充攻击成功力案例

- 2003年，结合时序攻击，破解IMAP电邮服务器与Outlook客户端之间的SSL/TLS加密信道，盗取用户密码。
- 2004年，证明攻击适用于ISO CBC模式加密和填充标准，可以有效地破译出明文。
- 2007年，攻击只加密不认证的IPsec网络配置，成功破解有效载荷（payload）明文。
- 2009年，利用消息认证码（MAC）执行时间的微小差别区分填充验证结果，攻破OpenSSL和GnuTLS的DTLS实现，恢复全部明文。
- 2010年，成功攻破验证码（CAPTCHA）系统，为后续拒绝服务攻击扫清道路；破译JavaServer Faces加密的访问状态；应用衍生的CBC-R加密算法，伪造可被服务器解密及填充验证无误的访问状态。
- 2011年，发现微软ASP.NET网页应用框架中的多个安全设计漏洞，可被攻击者利用破解未认证的加密小信息块（cookie）及伪造访问状态用以盗取服务器上限制访问目录中的文件。这给当时25%的互联网站点的安全性带来巨大冲击。
- 2012年，攻击硬件辅助RSA或CBC加密设备如USB安全令牌、智能卡和电子ID卡等，破解导入的密码。

2.4 填充攻击成功力案例

- 2013年，全新变种“幸运十三攻击”出现，其针对TLS协议中的 HMAC-SHA1 MAC验证的软件实现进行侧信道时序攻击。开源TLS/DTLS软件库包OpenSSL、GnuTLS、PolarSSL、CyaSSL及亚马逊AWS所开发的s2n都需要打补丁修复相应的漏洞。
- 2014年，谷歌安全团队报告“贵宾犬漏洞”（POODLE）。攻击者可利用软件实现中为照顾不同版本互通性而牺牲安全性的机制， 采用中间人攻击方式将客户端连接降级到SSL 3.0，再进行密文填充攻击破解密文。几个月后，又出现了一个新变种，攻击TLS 1.0-1.2服务器协议实现中的一些漏洞。
- 2016年，Steam游戏平台被曝光其客户端连接存在严重安全漏洞，可让攻击者组合重放攻击与密文填充攻击破解用户密码和其它保密信息；同年，常见漏洞与披露报告CVE-2016-2107揭示了OpenSSL针对“幸运十三攻击”的补丁中引入了另一个CBC模式密文填充攻击漏洞。

内 容 Contents

1 经典密码模式

2 CBC模式安全

3 扩展密码模式

4 序列密码初步

3.1密码反馈(CFB)模式

- CFB模式或OFB模式可将DES转换为流密码

- 加密：输入是64比特移位寄存器，其初值为某个初始向量IV
- 解密：将收到的密文单元与加密函数的输出进行异或
- CFB与CBC的区别是反馈的密文不是直接与明文相加，而是反馈至密钥产生器

$$Encryption: C_i = P_i \oplus SelectLeft_r\{E_k[ShiftLeft_r(S_i$$

$$Decryption: P_i = C_i \oplus SelectLeft_r\{E_k[ShiftLeft_r(S_i$$

E : Encryption　　D : Decryption　　S_i: Shift register
P_i: Plaintext block i　　C_i: Ciphertext block i　　T_i: Temporary register
K: Secret key　　IV: Initial vector (S_1)

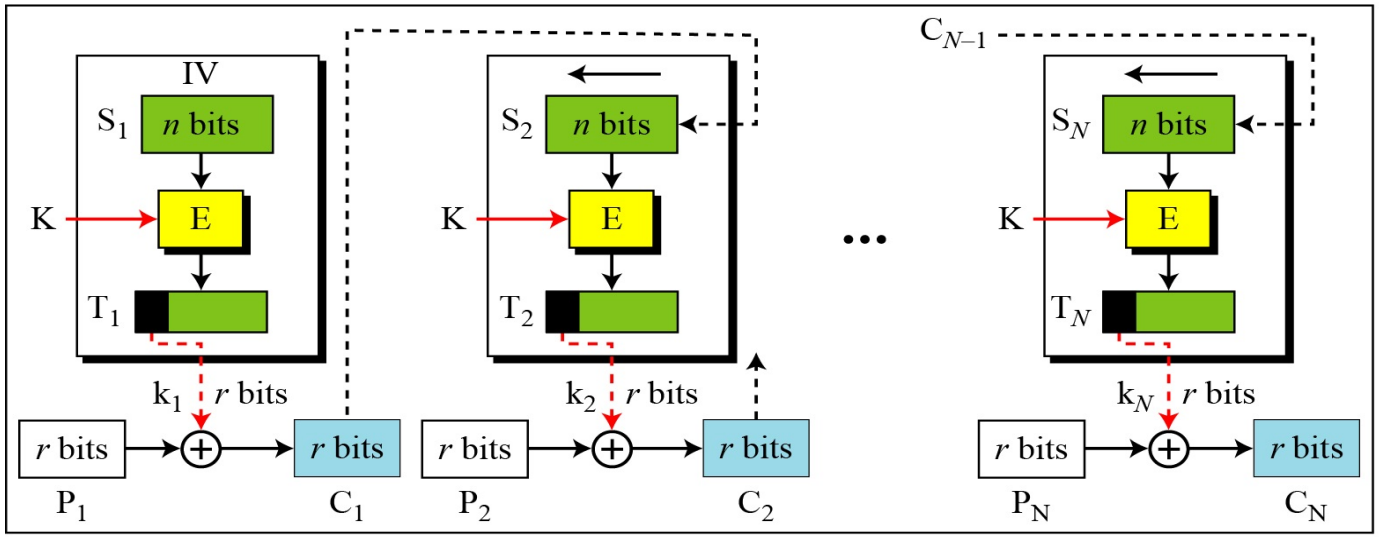


Figure 4 *Encryption in cipher feedback (CFB) mode*

3.1密码反馈(CFB)模式

- CFB的优点
  - 适于用户数据格式的需要。
  - 能隐蔽明文数据图样，也能检测出对手对于密文的篡改。

- CFB的缺点
  - 对信道错误较敏感，且会造成错误传播。
  - CFB也需要一个初始矢量，并要和密钥同时进行更换。
  - 无法同时并行处理多个明文分组

3.2 输出反馈(OFB)模式

- CFB模式或OFB模式可将DES转换为流密码

- 加密：将分组密码算法作为一个密钥流产生器，其输出的k-bit密钥直接反馈至分组密码的输入端，同时这k-bit密钥和输入的k-bit明文段进行对应位模2相加。

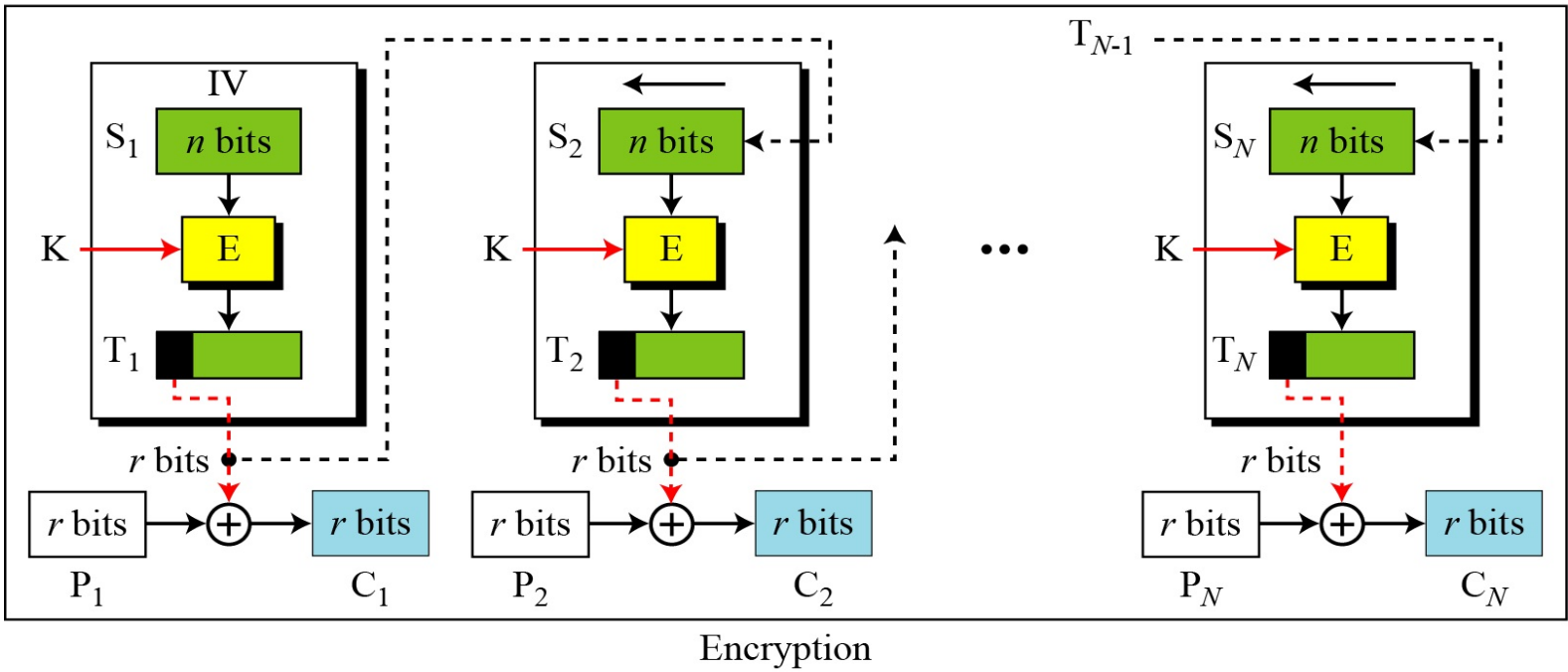


Figure 6 *Encryption in output feedback (OFB) mode*

3.2 输出反馈(OFB)模式

- 克服了CBC和CFB的错误传播所带来的问题
- 对于密文被篡改难以进行检测
- 不具有自同步能力，要求系统要保持严格的同步

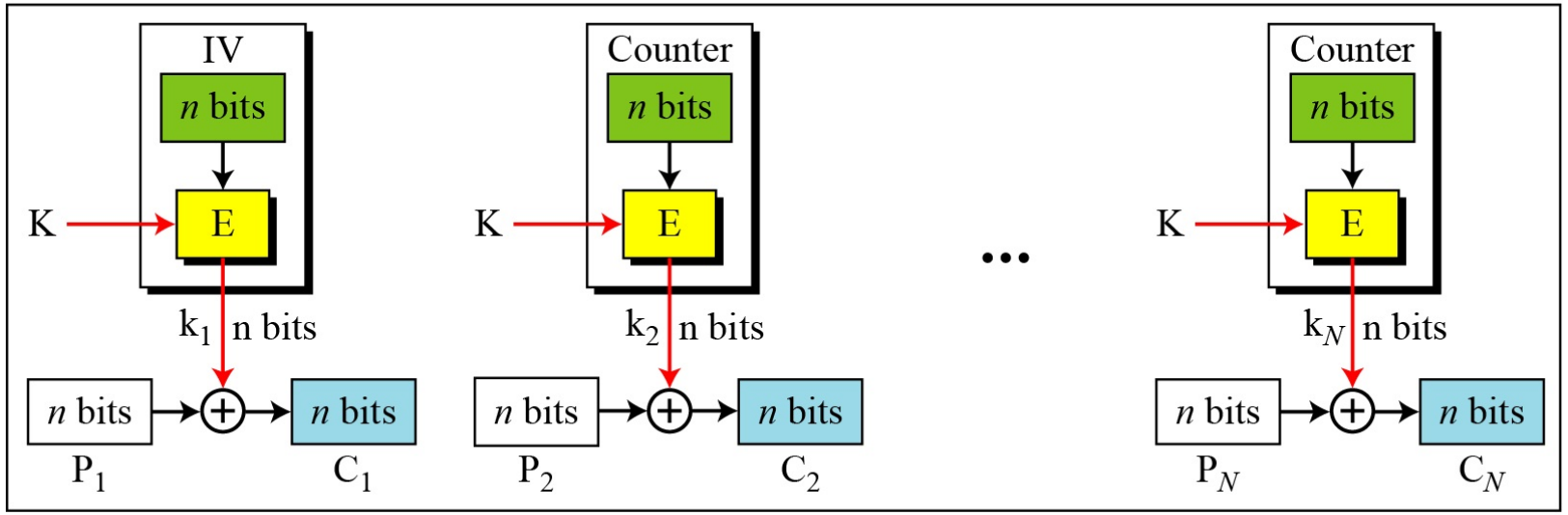
3.3 计数器(Counter)模式

- CTR模式被广泛用于ATM网络安全和IPSec应用中

Figure 8 *Encryption in counter (CTR) mode*

E : Encryption
P_i : Plaintext block *i*
K : Secret key
IV: Initialization vector
C_i: Ciphertext block *i*
k_i : Encryption key *i*

The counter is incremented for each block.



Encryption

3.3 计数器(Counter)模式

- 硬件效率：
  CTR 模式能够并行处理多个明文（或密文）分组的加密（或解密)。

- 软件效率：
  因为在 CTR 模式中能够进行并行计算，所以能充分利用支持并行功能的各类处理器，如流水线、每个时钟周期分派多个指令、大量寄存器和SIMD指令等。

- 预处理：
  基本加密算法的执行并不依赖于明文或密文的输入。因此，内存足够且能保证安全时，可以首先采用预处理来准备加密盒的输出，并将输入送入XOR函数。

3.3 计数器(Counter)模式

- 随机访问：
  明文或密文的第i个分组能以随机访问的方式处理。

- 简单性：
  与ECB和CBC模式不同，CTR 模式只要求实现加密算法，而不要求实现解密算法。

- 无填充：
  由于CTR模式的结构，不需要对明文进行补充。当最后一个明文分组不是完整的分组时（如长度为u位不完整的分组），它的最高u位用来进行异或运算，其余b-u位责备丢弃（b为分组长度）。

3.4 随机数的特点

- **均匀分布**：数列中每个数出现的频率应相等或近似相等；
- **独立性**：数列任意一数都不能由其它数推出。

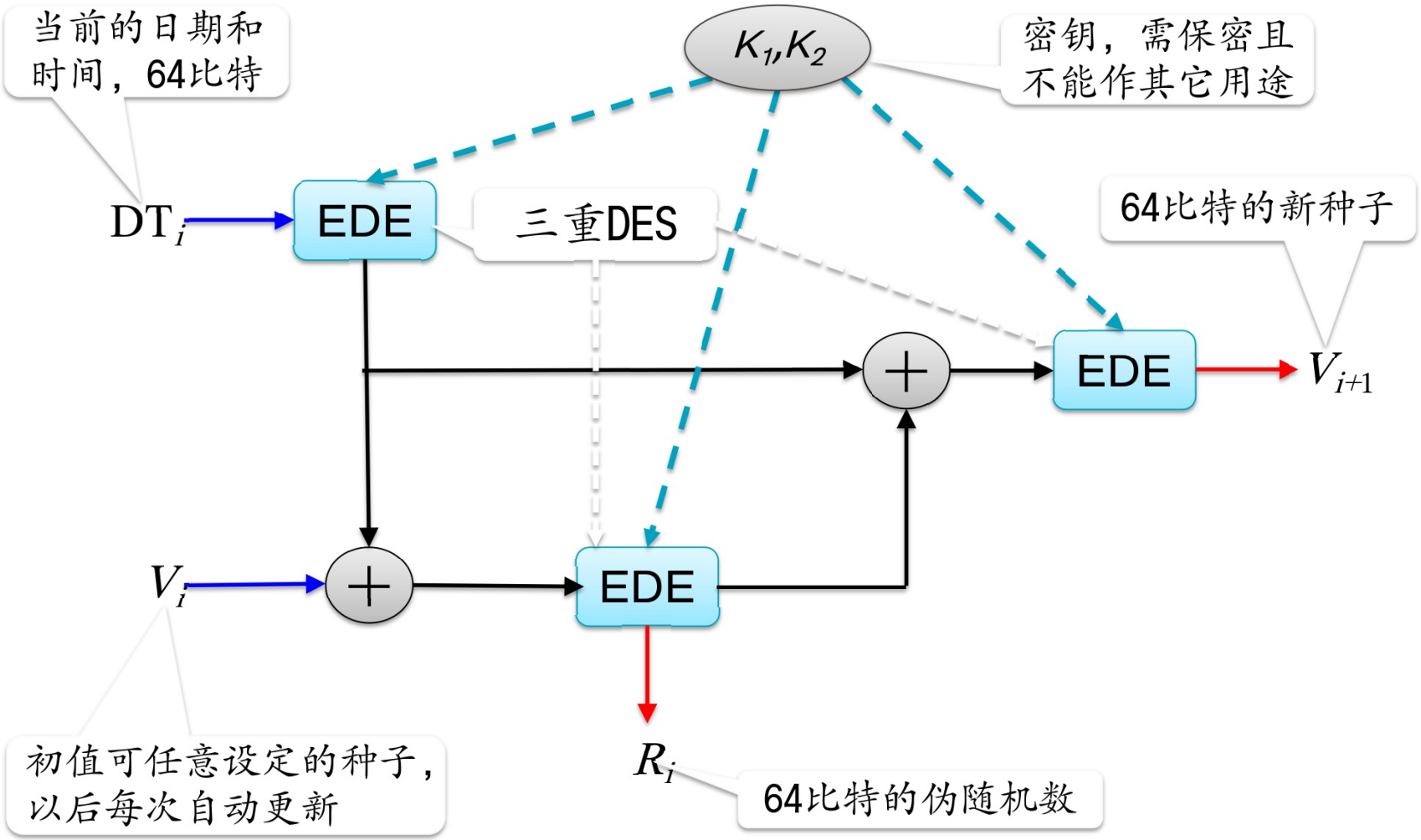
备注：

目前实际应用中所需的随机数都借助于安全的密码算法来产生的，但由于算法是确定性的，因此，严格上讲，产生的数列不是随机的，然而如果算法设计得好，产生的数列就能通过各种实际的随机性检验，这种数就是**伪随机数**。

3.4 ANSI X9.17简介

- ANSI X9.17-1985 是金融行业密钥管理的奠基性标准，由美国国家标准协会发布，旨在规范 DES/3DES 密钥的全生命周期管理。其设计思想影响了后续众多安全协议，但技术实现已被 NIST SP 800-90A 等现代标准替代，FIPS 171 于2005年撤回标志着其历史使命的阶段性完成
- 核心规范 DES 与三重 DES 密钥的生成、交换、使用、存储和销毁全过程，为金融交易安全提供标准化框架。其密钥分层架构设计思想至今仍被现代密码学标准所借鉴和沿用

3.4 ANSI X9.17的伪随机数产生器



3.4 ANSI算法评价

本方案具有非常高的密码强度，这是因为采用了112比特长的密钥和9个DES加密，同时还由于算法是由两个伪随机数输入驱动，一个是当前的日期和时间，另一个是算法上次产生的新种子。

而且即使某次产生的伪随机数R_i泄露了，但由于R_i又经一次EDE加密才产生新种子V_{i+1}，所以别人即使得到R_i也得不到V_{i+1}，从而得不到新伪随机数R_{i+1}。

Contents 内 容

1 经典密码模式

2 CBC模式安全

3 扩展密码模式

4 序列密码初步

4.1 序列密码起源

- 维尔姆(Vernam)密码
  - ✓ 序列密码通常认为起源于上个世纪20年代的维尔姆(Vernam)密码
  - ✓ Vernam密码中的密钥序列要求是随机的序列，由于随机的密钥序列的产生、存储以及分配等方面存在一定的困难，Vernam体制在当时并没有得到广泛的应用
- 基于伪随机序列的序列密码
  - ✓ 随着微电子技术和数学理论的发展和完善，基于伪随机序列的序列密码得到了长足的发展和应用，其产生有比较成熟的数学理论工具
- 序列密码与“一次一密”
  - ✓ 如果密钥序列是随机的序列，则序列密码就是“一次一密”密码体制，商农已经证明“一次一密”密码体制在理论上是不可破译的
- 其他
  - ✓ 序列密码又称为流密码，属于对称密码
  - ✓ 序列密码是世界各国的军事和外交等重要领域中使用的主要密码体制之一

4.2 序列密码-解析表达

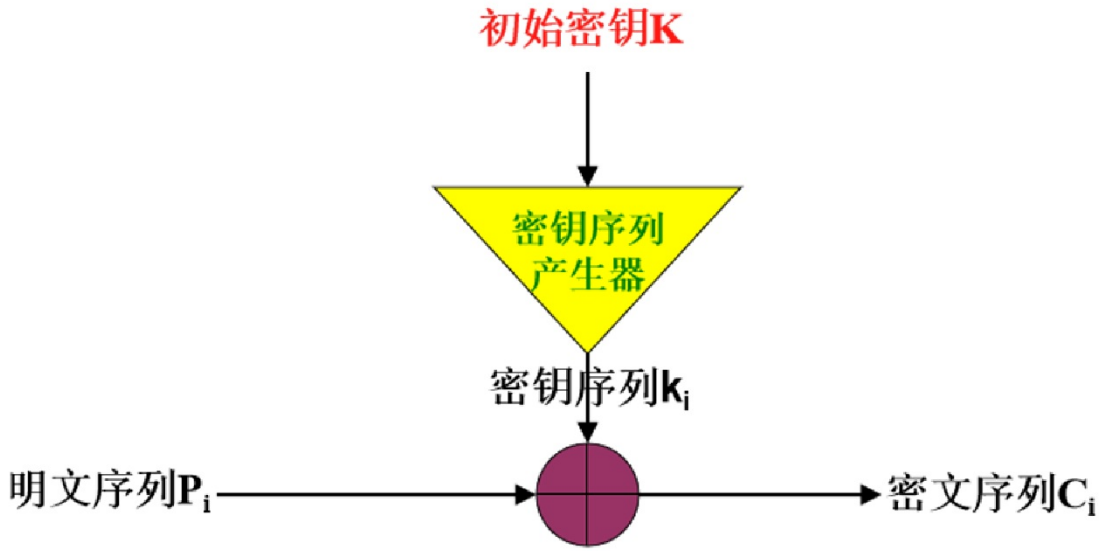
$$C = P \oplus K$$

$$C \oplus K = P \oplus K \oplus K = P$$

在序列密码中，将明文消息按一定长度(长度较小，譬如1个比特)分组，然后对各组用“随机”的密钥序列进行加密，产生相应的密文。解密用相同的密钥序列对密文序列进行分组解密以恢复出明文序列。

- ✓ 设明文为$p=p_0p_1p_2\ldots$ $p_i \in GF(2)$, $i \geq 0$
- ✓ 设密钥为$k=k_0k_1k_2\ldots$ $k_i \in GF(2)$, $i \geq 0$
- ✓ 设密文为$c=c_0c_1c_2\ldots$ $y_i \in GF(2)$, $i \geq 0$
- ✓ 则加密变换为$c_i = E_{ki}(p_i)$ $i \geq 0$
- ✓ 则解密变换为$p_i = D_{ki}(c_i)$ $i \geq 0$

4.2 序列密码-简图

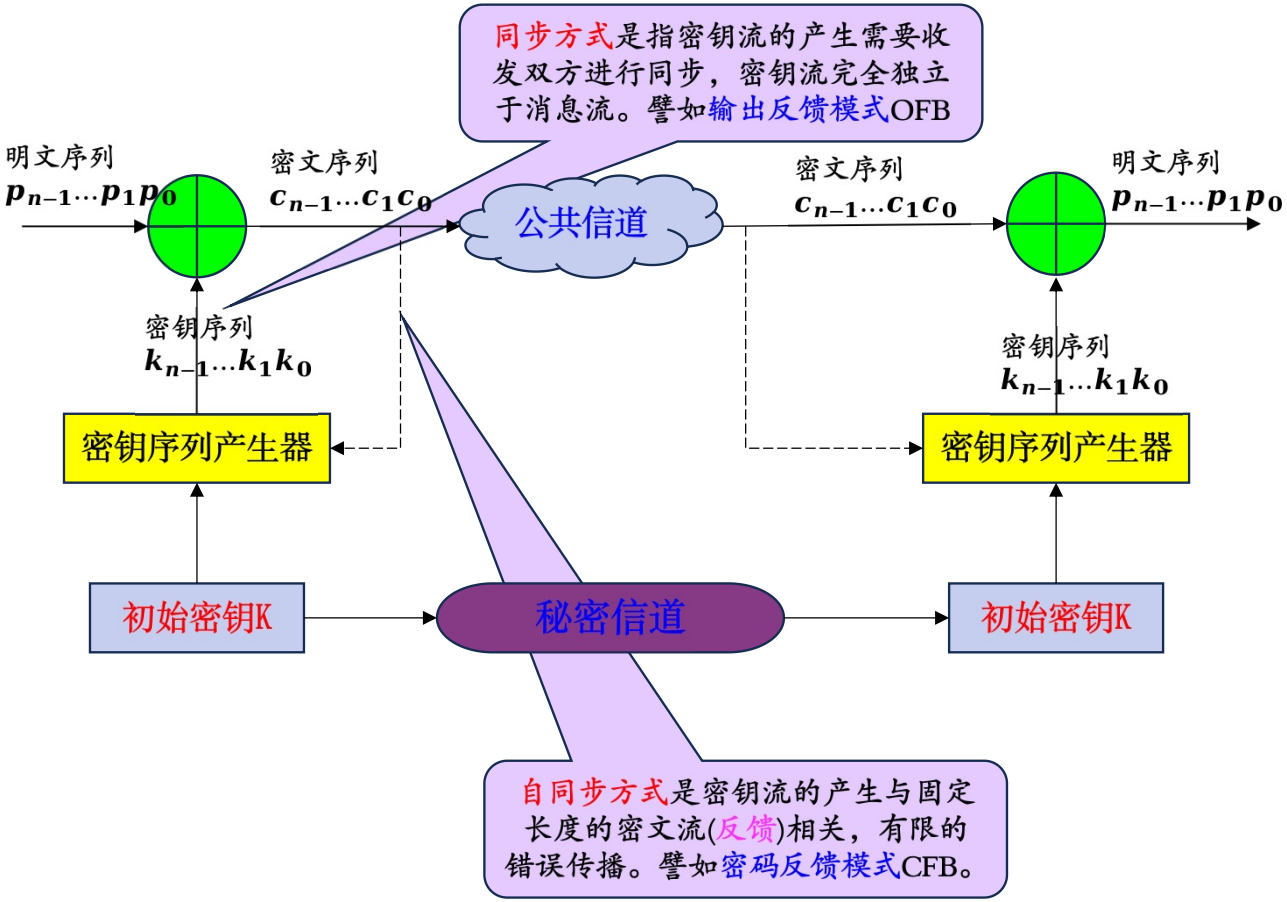


特点：

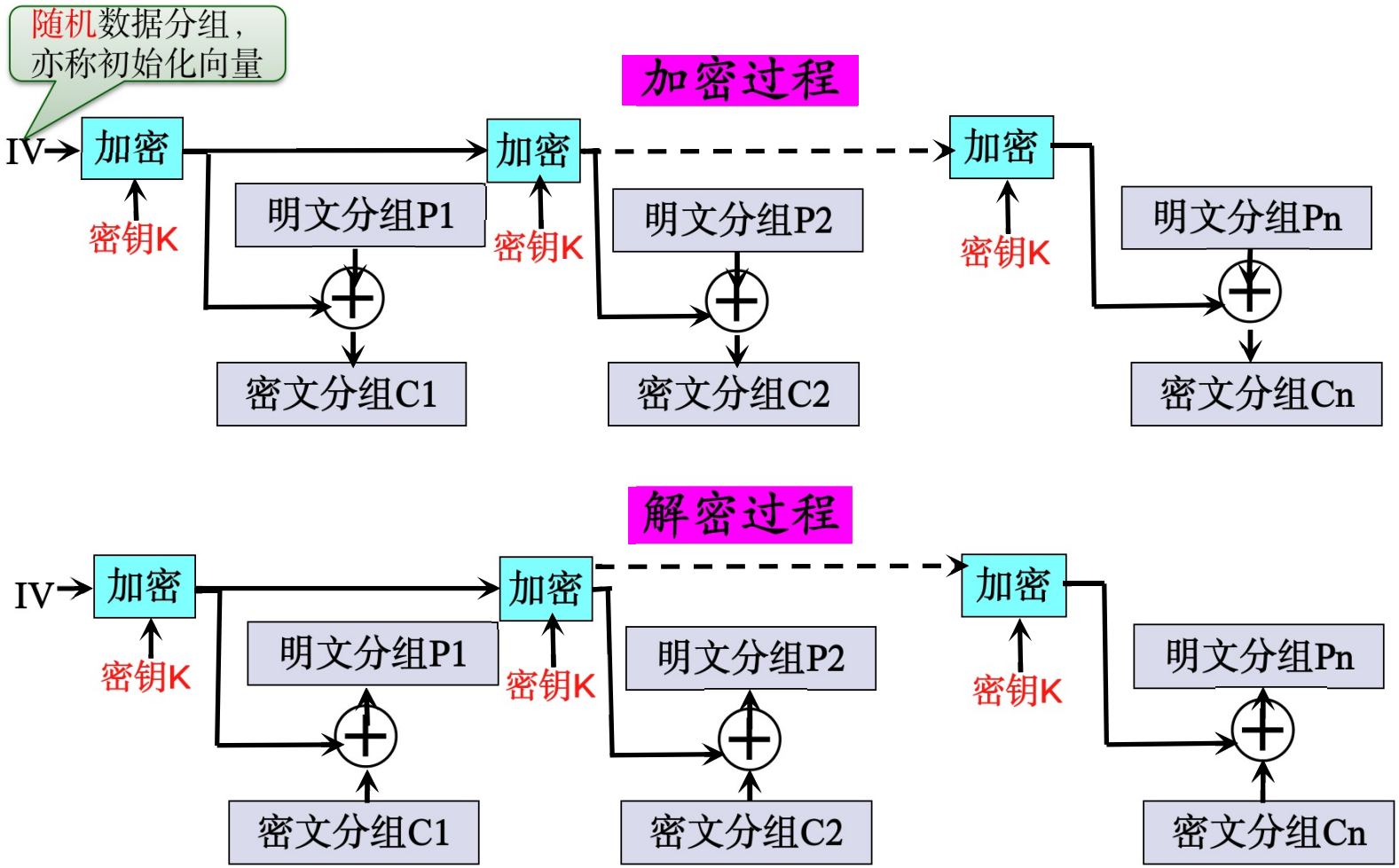
√加解密运算只是简单的模二加(异或)运算。

√密码安全强度主要依赖密钥序列的安全性。

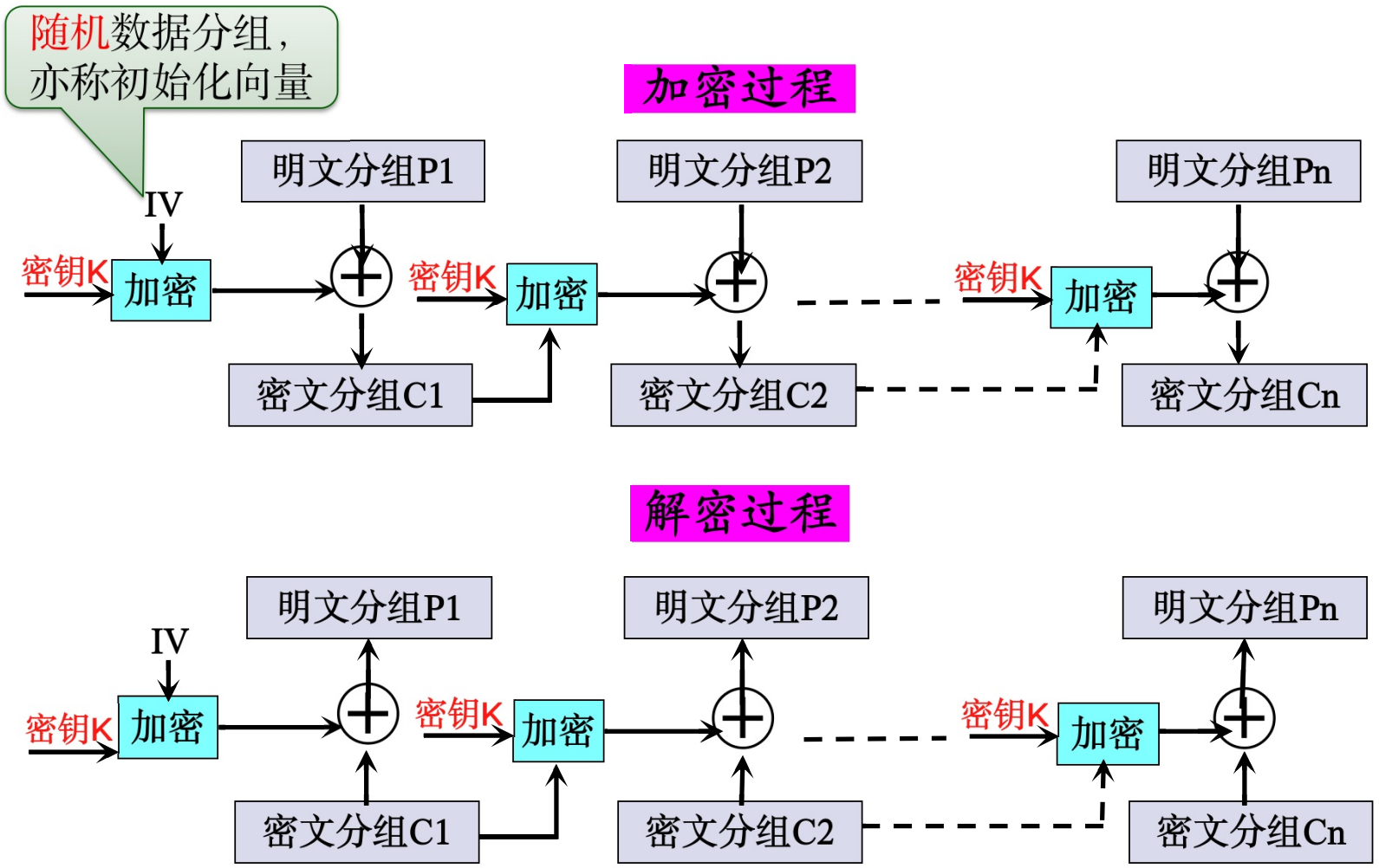
4.2 序列密码-应用模型


同步方式是指密钥流的产生需要收发双方进行同步，密钥流完全独立于消息流。譬如输出反馈模式OFB
明文序列
$p_{n-1}\cdots p_1 p_0$
密文序列
$c_{n-1}\cdots c_1 c_0$
公共信道
密文序列
$c_{n-1}\cdots c_1 c_0$
明文序列
$p_{n-1}\cdots p_1 p_0$
密钥序列
$k_{n-1}\cdots k_1 k_0$
密钥序列
$k_{n-1}\cdots k_1 k_0$
密钥序列产生器
密钥序列产生器
初始密钥K
秘密信道
初始密钥K
自同步方式是密钥流的产生与固定长度的密文流(反馈)相关，有限的错误传播。譬如密码反馈模式CFB。

4.3 基于分组密码的算法-输出反馈模式（OFB）



4.3 基于分组密码的算法-密文反馈模式（CFB）



4.4 密钥序列产生器(KG)基本要求

- 初始密钥K的长度足够长，一般应在128位以上(抵御穷举攻击)；
- 密钥序列产生器KG生成的密钥序列$\{k_i\}$具极大周期；
- 密钥序列$\{k_i\}$具有均匀的n-元分布，即在一个周期内，某特定形式的n-长bit串与其求反，两者出现的频数大抵相当；
- 由密钥序列$\{k_i\}$提取关于种子密钥K的信息在计算上不可行；
- 雪崩效应。即种子密钥K任一位的改变要引起密钥序列$\{k_i\}$在全貌上的变化；
- 密钥序列$\{k_i\}$不可预测的。密文及相应的明文的部分信息(确定部分密钥序列)，不能确定整个密钥序列$\{k_i\}$。

小结

	优点	缺点
ECB	简单 可并行计算 误差不传递	不能隐藏明文模式 主动攻击
CBC	不容易主动攻击 适合长报文	无法并行 误差传递
CFB	不容易主动攻击 分组转变为流模式 可加密小于分组数据	无法并行 误差传递
OFB	分组转为流模式 可加密小于分组数据	主动攻击
CTR	并行 一次一密	主动攻击

课后

- 1、描述Padding Oracle Attack破解密文过程，尝试利用AI做视频?
- 2、描述基于初始化向量的cookie恢复原理?
- 2、列举不同分组密码模式应用场景，每种模式列举两种。

谢 谢